



INSTALLATION PLANNING GUIDE FOR K2VIEW FABRIC SERVICES ON AMAZON AWS

K2view Data Product Platform Services

Version: 1.6

Date: 03 November 2025

This document contains copyrighted work and proprietary information belonging to K2view. All intellectual property rights (including, without limitation, copyrights, trade secrets, trademarks, etc.) evidenced by or embodied in and/or attached, connected, or related to this document, as well as any information contained herein, are and shall be owned solely by K2view. K2view does not convey to you an interest in or to this document, the information contained herein, or its intellectual property rights, but only a personal, limited, fully revocable right to use the document solely for review purposes. Unless explicitly set forth otherwise, you may not reproduce by any means any document and/or copyright contained herein.

Information in this document is subject to change without notice. Corporate and individual names and data used in the examples herein are fictitious unless otherwise noted.

Copyright © 2025 K2view Ltd. / K2VIEW LLC. All rights reserved.

The following are trademarks of K2view: K2view logo, K2view's platform.

K2view reserves the right to update this list document from time to time.

Overview

This document is a planning guide for installing K2view Fabric Services on Amazon AWS. It details all necessary steps, prerequisites, and configurations to ensure a successful deployment. It is designed for IT professionals, system administrators, and project owners who are involved in implementing K2view Fabric's Data Product Platform on AWS infrastructure.

The guide outlines the process, from initial planning and prerequisite validation to provisioning resources, configuring security settings, and populating your container registry (e.g., [Amazon ECR](#)) and TLS certificates. It includes a checklist and worksheet to capture essential information required at each stage, making the process more manageable and organized.

Following this guide will help you prepare for deploying K2view Fabric Services.

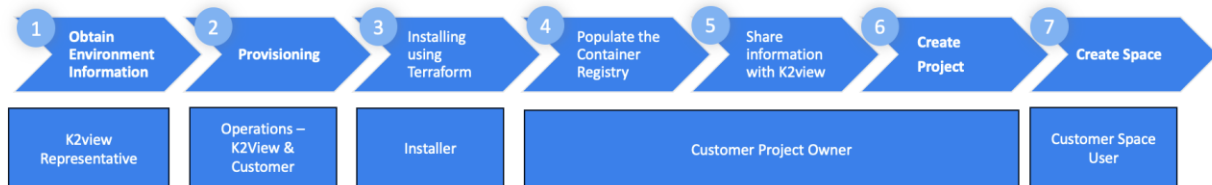
Table of Contents

Overview.....	2
1. Planning and Installation Step Overview	4
Step 1 – Obtaining Environment Information	4
Step 2 – Provisioning	4
Step 3 – Install the K2view Fabric Services.....	4
Step 4 – Populate the Container Registry with K2view Docker images.....	4
Step 5 – Share Container Registry and Domain Name information with K2view	5
Step 6 – Create the K2view Project	5
Step 7 - Create a K2view Space	5
2. Provisioning	6
2.1. Information Required by K2view – Before the Installation.....	6
2.2. K2view Provisioning	6
2.3. Information Required by K2view – Before Proceeding to Step 5	7
2.4. Persistent Storage Selection	7
3. Prerequisites	9
3.1. Internet Access	9
3.2. Whitelisting	9
3.3. DNS	9
3.4. TLS keys and certificates.....	10
3.5. Permissions and Roles	10
3.6. Tools.....	13
3.7. Git code repository.....	13
3.8. Container Registry.....	13
4. Installation Checklist Worksheet.....	15
5. Instructions Details.....	19
5.1. AWS Privileges	19
5.2. IAM Policy Permissions.....	23
5.3. Cloning the K2view Blueprints	25
5.4. Configuring Your Deployment.....	25
5.5. Populating Your Container Registry	26
5.6. Sign In to K2cloud.....	28
5.7. Create the Project	28
5.8. Creating Your First Fabric Space	31

1. Planning and Installation Step Overview

Planning and capturing the necessary information before installing K2view Fabric Services on Amazon AWS will help ensure a smooth installation experience. The customer and K2view will gather and share information used to provision the environment.

The Installation steps are:



Step 1 – Obtaining Environment Information

A K2view representative working with the customer will discuss the prerequisites to complete and review the [Prerequisites](#) section and the [Installation Checklist Worksheet](#).

This review should be used to determine the components to be provisioned based on your use case.

Step 2 – Provisioning

K2view Provisioning

K2view will create a Container registry account, user accounts, and associated configurations for you. Some of the information needed to carry out this function will be obtained during Step 1. Other information K2view needs to complete your environment's configuration will need to be provided by you during Step 5.

Customer Preparation

You will need to gather the information described in the [Prerequisites](#) section. You can use the [Installation Checklist Worksheet](#) to capture this information. This includes Git details, obtaining a TLS certificate (PEM encoded) and its private key (PEM encoded), and the names of entities. The certificate and private key files need to be base64 encoded. You must also review what is required to perform the installation. These are listed in the [Prerequisites](#) section.

Step 3 – Install the K2view Fabric Services

The installation will be run by someone with the necessary [roles and privileges](#) within your AWS account. These instructions are provided in a separate installation guide.

Step 4 – Populate the Container Registry with K2view Docker images

The K2view project owner will load the selected Container Registry (e.g., [Amazon ECR](#)) with K2view Docker images identified by your K2view representative during Step 1.

Step 5 – Share Container Registry and Domain Name information with K2view

The Customer Project Owner must share the following information to complete the configuration for the tenant's creation of Projects and Spaces.

- The fully qualified domain name (FQDN) used for your Domain Ingress
- The list of Container Registry locations of your Docker images

This information may have been identified during pre-planning, but the specific information should be confirmed to avoid misconfiguration. We recommend that you track this information using the [Installation Checklist Worksheet](#).

Step 6 – Create the K2view Project

To create the K2view Project, refer to the [Creating Your Fabric Project](#) section.

Step 7 - Create a K2view Space

To create your K2view Space, refer to the [Creating Your First Fabric Space](#) section.

2. Provisioning

2.1. Information Required by K2view – Before the Installation

Before the installation. K2view needs information to provision your environment.

- **First Admin user:** the user that will set up the K2view project and create the first space. Additional user accounts can also be created.

The information required for users includes:

- Name
 - Email address
 - Phone number with country code included
-
- **Your Cloud provider** used for the installation, in this case, Amazon AWS
 - **A site name and purpose** (could be something like POT, DEV, NPRD, PROD). This information will be used by K2view to better differentiate your first site from others you might create in the future.
 - **The domain name of your Kubernetes Ingress Controller.** If not available before the installation, it can be provided before proceeding to Step 5. You will need this domain name to create your Ingress Controller, register a DNS A record, and create a certificate to secure access to your K2view spaces.

2.2. K2view Provisioning

K2view will provision several things for you:

- Your tenant and its configurations
- Your initial K2view Admin user
- A K2view Container registry account for you to access K2view images
- A K2view Cloud Mailbox ID for you to connect to the K2cloud Orchestrator service

K2view will also provide you with additional information, such as:

- In consultation with your K2view representative, Docker images and their associated versions will be recommended. You will need this information to use docker pull commands to obtain images such as fabric, fabric-studio, and the k2-cloud-deployer images.
- K2view may also provide you access to k2exchange.k2view.com, depending on your needs and those identified by your K2view representative.

2.3. Information Required by K2view – Before Proceeding to Step 5

Before you proceed to Step 5, you must pause to provide K2view with information that will allow it to finalize the configuration of your environment. K2View needs:

- The location of the images you loaded to your container registry (e.g., [Amazon ECR](#)) from the K2view Container registry.
 - For example, if you pushed fabric-studio:8.2.1_40 to your container registry, you will need to provide K2view with its location value in the form of [yourRegistry]/k2view/fabric-studio:8.2.1_40 (or the location you configured)
- We need to know the specific images (e.g. fabric-studio) and versions (e.g., 8.2.1_40) you used. These will be recommended by a K2view representative.
- When configuring your Kubernetes nodes, the type of storage – a Storage class – is needed by K2view to configure your environment.
 - If you are using an existing cluster, we need to learn if you used EBS or EFS.
 - If you use K2view's [AWS Blueprint](#), what you configured is also needed (EBS or EFS).

2.4. Persistent Storage Selection

When designing storage solutions for applications running in AWS, selecting the right type of persistent storage is important to ensuring availability, reliability, and scalability. AWS offers multiple options for persistent storage, including Amazon Elastic Block Store (EBS) and Amazon Elastic File System (EFS). Each solution is tailored to specific use cases and operational needs, and understanding their pros and cons is essential for optimal implementation. The following subsections explore the limitations and benefits of EBS and EFS, guiding when to choose one over the other.

EBS and Its Limitations

Amazon Elastic Block Store (EBS) provides high-performance block storage for single availability zone (AZ) use cases, but its AZ-bound nature presents challenges in multi-AZ environments.

1. **AZ-Bound Nature.** Amazon Elastic Block Store (EBS) volumes are tied to a specific AZ. If a pod using an EBS volume is terminated and rescheduled in another AZ, it cannot access the original volume because it cannot be mounted across AZs.
2. **Persistent Pod Needs.** Your pod needs a persistent storage layer to retain data beyond its lifecycle. EBS can meet this requirement **only if the pod remains in the same AZ** where the EBS volume exists.
3. **Pod Scheduling and Multi-AZ Clusters.** In Kubernetes clusters running in EKS, pods can be rescheduled in any AZ within the cluster. If a pod moves to another AZ, the storage layer (EBS volume) becomes inaccessible, causing a failure to retrieve persistent data.

Why Consider EFS

Amazon Elastic File System (EFS) provides a flexible and scalable storage solution for workloads that demand cross-AZ accessibility, shared access, persistent storage, and seamless scalability.

1. **Cross-AZ Access.** EFS is a **managed NFS (Network File System)** accessible across all AZs within the same region. It allows any pod, regardless of its AZ, to access the shared filesystem seamlessly.
2. **Shared Access.** EFS supports multiple clients accessing the filesystem simultaneously, making it ideal for workloads where multiple pods might need to read or write from the same storage.
3. **Persistence Across Pod Lifecycle.** EFS ensures data remains persistent even if the pod is terminated and recreated in a different AZ.
4. **Scalability.** EFS automatically scales as your data grows, eliminating the need for manual resizing.

When Should You Use EFS Over EBS

You should choose EFS when:

- You are required to run in a **multi-AZ EKS cluster**.
- Your pods may be rescheduled across different AZs.

3. Prerequisites

Specific prerequisites must be met, and planning steps must be taken to install K2view Fabric Services, populate your container registry (e.g., [Amazon ECR](#)), and create a K2view Project and a Space.

3.1. Internet Access

K2view Software

1. The installation presumes that you have Internet access from which to obtain Fabric images from the K2view Container registry and perform a Git clone to your machine
2. You need a K2view Container registry account to obtain a Fabric Studio docker image. Your K2view account representative can arrange this for you. It is required as part of Step 4 of the installation.

Internet Access Required

Internet access is required to perform this installation. You will need access to:

1. Github.com to clone K2view's blueprints at <https://github.com/k2view/blueprints.git>
2. K2view's Container registry Docker Image repository at <https://docker.share.cloud.k2view.com>
3. If you plan to install TDM, you will need access to K2view's Exchange

3.2. Whitelisting

Certain functions require HTTPS access over port 443:

- **K2view Agent** – makes outbound REST calls to <https://cloud.k2view.com> to obtain deployment and configuration instructions via the K2cloud Orchestrator's Mailbox service
- **For Installations and Updates** –
 - <https://share.cloud.k2view.com> to fetch K2view Fabric, Web Studio and k2-agent images
 - <https://github.com> to pull K2view blueprints
 - <https://github.com> to pull K2view deployment Helm charts

You may need to whitelist data sources/targets and open HTTPS ports to meet your needs.

3.3. DNS

Using [Context-Based URLs](#) for your spaces allows you to define a single DNS A record. This is available with Fabric 8.2 and later. Before Fabric 8.2, a wildcard certificate and domain needed to be provisioned.

The Terraform configuration creates an Azure DNS Zone by default. It will configure everything you need to get up and running.

If you use a preexisting DNS zone, you need this domain name to create your Ingress Controller and register a DNS A record. You must create a DNS record pointing to the node hosting the K8s Ingress Controller (it can point to a private or a public IP). The DNS record for the server hosting the K8s node does not need to be registered in the public DNS. The only requirement is that users can resolve the DNS name internally to the customer's environment.

DNS Configuration

Please use AWS Route53 to create and manage DNS records and create the following DNS record:

Type: A (or CNAME depending on your DNS setup)
Name: example.com
Value: NLB DNS name (e.g., nlb-abc123.amazonaws.com)

3.4. TLS keys and certificates

When using K2view's [AWS Blueprint](#) and Terraform, a TLS PEM certificate and its associated TLS PEM Key file are needed. The NGINX Controller will use this certificate to terminate TLS connections.

Your network team must create a TLS certificate for the domain you selected as your Domain Ingress. The entire certificate authority chain needs to be present. During the installation, you must specify the path to your PEM TLS certificate and its associated private key.

Please note that for a Proof of Technology (POT) environment, both domain and certificates can be provided by K2view should this be useful to you.

3.5. Permissions and Roles

Permissions and roles are required by the user performing the installation. AWS account with AWS Administrator permissions are described in the [AWS Privileges](#) section.

Please ensure a client environment with the following command-line tools is configured: aws-cli, kubectl, and helm (see the [Tools](#) section).

Additionally, you need a service account or user with admin access to a namespace in the target Kubernetes cluster. For namespaces handling large-scale data (e.g., a production namespace in a Fabric cluster), it is recommended to use managed services such as **managed Postgres** or **bucket/blob storage**. During the namespace creation process, K2Cloud dynamically provisions the required managed resources. The k2-agent namespace must be configured with appropriate credentials to facilitate this process.

You can use AWS's Kubernetes cloud-native authentication mechanism to provide these credentials. Assign an IAM role ARN to the k2view-agent namespace service account. This IAM role should be configured in the k2-agent settings.

IAM Role Creation

Upon creating a space, the K2View Agent creates a Kubernetes "Deployer" Job that runs and creates the requested AWS cloud resources depending on the Space profile defined in the Cloud Orchestrator. For each space, this might include the following based on your space profile:

- S3 Bucket
- RDS Postgres Database
- Keyspaces

For each of the above resources, the "Deployer" job provisions the requested resources and utilizes AWS IRSA (IAM Roles for Service Accounts) to establish synchronization between the AWS resource Administrator IAM role and the pods running within the newly created namespace, allowing access to the associated AWS resources. We don't specify Worker node permissions as each space may be assigned its own IAM role and policy depending on its profile.

Permissions by Service. Here is a summary of the permissions required by service - see details in the [AWS Privileges](#) section

IAM Permissions: For managing roles and policies related to EKS.

- Includes creating, attaching, and managing IAM roles and policies (e.g., iam:CreateRole, iam:AttachRolePolicy, iam:PassRole).

EKS Permissions: For EKS cluster and node group operations.

- Includes creating and managing clusters and node groups (e.g., eks:CreateCluster, eks:DescribeNodegroup).

EC2 Permissions: For networking and instance management.

- Includes managing VPCs, subnets, security groups, and EC2 instances (e.g., ec2:CreateVpc, ec2:RunInstances).

Elastic Load Balancer (ELB) Permissions: For managing load balancers used by Kubernetes services.

- Includes creating and managing load balancers and target groups (e.g., elasticloadbalancing:CreateLoadBalancer).

Auto Scaling Permissions: For managing worker node auto-scaling.

- Includes creating and updating auto-scaling groups (e.g., autoscaling:CreateAutoScalingGroup).

CloudWatch Permissions: For monitoring and logging EKS events.

- Includes creating and managing log groups and streams (e.g., logs:CreateLogGroup, logs:PutLogEvents).

STS Permissions: For role assumption and authentication.

- Includes assuming roles via sts:AssumeRole.

The generated AWS IAM Policy associated with *-fabric IAM Role is described in the [IAM Policy Permissions](#) section.

Kubernetes Permissions and Roles for Agent Deployment

In addition to AWS permissions, Kubernetes cluster-level permissions are required to deploy the K2view Agent and supporting resources into the EKS cluster.

The K2view Agent is deployed into its namespace (typically k2view-agent) using a Helm chart. The Helm deployment process creates Kubernetes objects including:

- Namespace
- Service Account
- Role and RoleBinding
- Deployments
- Secrets and ConfigMaps

The user performing the Helm-based agent installation must have Kubernetes permissions to create these resources. Specifically:

Resource	Verbs
namespaces	create
serviceaccounts	create, patch, delete, get, list
roles / rolebindings	create, patch, delete, get, list
deployments / statefulsets / jobs	create, patch, delete, get, list
secrets / configmaps	create, patch, delete, get, list

The Helm chart may optionally configure workload identity via IRSA by associating an AWS IAM role with the Agent Service Account.

Once installed, the K2view Agent will have elevated permissions within its own namespace. The Agent's Service Account is granted access to create and manage Kubernetes resources that support project, space, and runtime workloads under K2Cloud Orchestrator control.

3.6. Tools

The following tools are required to perform the installation by the user with the required [Permissions and Roles](#).

Please install the following tools:

- Terraform – version 1.9.x - [HashiCorp install guide](#)
- Kubectl – version 1.14.0 - [Kubectl install guide](#)
- AWS CLI – latest version - [Installing the AWS CLI](#)
- Helm – version 2.13.0 - [Helm install guide](#)
- Docker – latest version - You can use the tools provided by your container registry to pull from the K2view Container registry and push to your registry. Docker or an OCI-compatible tool will likely be needed for this.

3.7. Git code repository

A Git code repository is required for your Fabric Project. Three pieces of information will be required to create your K2view project including:

- Repository URL
- Git token
- Git Branch / Tag

3.8. Container Registry

K2view will create a Container registry account, user accounts, and associated configurations for you. You will use the K2view Container registry account to obtain K2view Docker images. In Step 5, K2view will require the location of the images—— you push to your container registry (e.g., [Amazon ECR](#)).

You must populate your selected container registry by pulling and pushing docker images. Docker must be installed on a computer to pull and push docker images from the K2view Container registry to the container created. You will need a K2view Container registry account you can obtain from your K2view representative

Outbound HTTPS/443 Access

You need to confirm outbound HTTPS access over port 443 for:

- The host “pulling” from K2view’s Container registry has outbound HTTPS access over port 443.
- Users have HTTPS access to the K2view Cloud Manager over port 443.

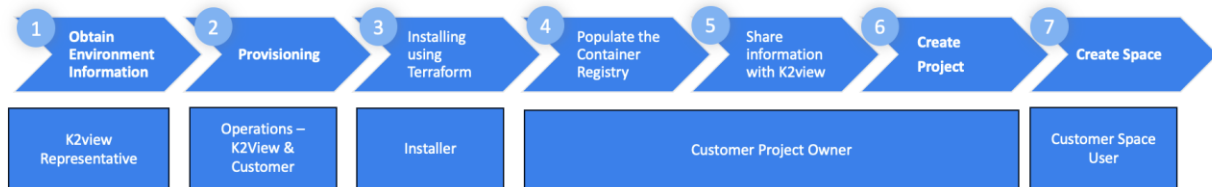
Extracting and pushing Docker images to your Container Registry

- You will need the K2view Container registry account to perform the “pull” action. This will be provided to you by your K2view representative.
- You will need access to your container registry to publish K2view and other Docker images.
- You must note the location of the required Docker Images posted to your container registry. K2view needs this information to complete your provisioning in Step 5.

4. Installation Checklist Worksheet

Use this section to gather the information you need before installing. For more information, please consult the [Prerequisites](#) section.

At various steps during the installation, you must obtain information from K2view and, in Step 5, provided information to K2view.



The following summarizes the information you and K2view require.

Assets you will need for your installation

As input to your [Terraform-based installation](#), you will need to have obtained:

- [] **Domain Ingress FQDN:** _____
Subdomain of your cluster. A DNS A record of site.domain.com will point to the load balancer IP.
- [] **TLS PEM certificate file:** A file
A full CA chain certificate for the Domain Ingress (e.g., fullchain.pem of site.domain.com)
- [] **Associated TLS PEM Private Key:** A file

You have confirmed Internet connectivity to specific locations

Certain functions require HTTPS access:

- [] **K2view Agent** – makes outbound REST calls to <https://cloud.k2view.com> (port 443) to obtain deployment and configuration instructions via the K2cloud Orchestrator's Mailbox service.
- [] **<https://share.cloud.k2view.com>** (port 443) to fetch K2view Fabric, Web Studio and k2-agent images.
- [] **<https://github.com>** (port 443) to pull K2view blueprints and deployment Helm charts.
- [] You may need to whitelist specific data sources/targets and open HTTPS ports as required to meet your needs.

Please share with K2view – Required for Step 2

Please share this with your K2view representative for this information.

[] K2view Admin user:

○ Name: _____

○ Email address: _____

○ Phone number with country code included: _____

Please request a K2view Container registry Account – Required for Step 3

Please request that your K2view representative provide you with an account for the **K2view Container registry**. This account is a shared account intended for your entire organization.

Shared by K2view – Required for Step 3

Once the above is shared with your K2view representative you will be provided the following information.

[] K2view Cloud Mailbox ID provided to you by K2view: _____

[] K2view Container registry Account supplied by K2view: _____

[] List of Docker images to pull from Container registry:

Recommended by K2view. Please consult with your K2view representative for this information.

After completing Step 4—the image population of your Container Registry—and before creating your K2view Project and your first Space, you must provide the following to K2view.

[] List of Docker images you pushed to your container registry:

Please share this information with your K2view representative. After you have populated your container registry, this list of locations must be shared with your representative K2view.

[] Domain Name of your Ingress Controller:

It can be shared with K2view before the installation if you have selected the domain

[] Cloud Provider:

This information will help K2view better understand your deployment.

[] A site name and purpose:

This optional information such as POT, DEV, NPRD, PROD will be used by K2view to differentiate your first site better from others you might create in the future

K2view Project Creation Step – Required for Step 6

You will need the following information to create your project in Step 6.

[] Git Repository URL: _____

[] Git token (do not share): _____

[] Git Branch / Tag: _____

5. Instructions Details

5.1. AWS Privileges

To successfully deploy and manage Amazon EKS clusters and their associated resources, it's essential to configure the appropriate permissions across multiple AWS services. These permissions ensure seamless operations for tasks such as cluster creation, node group management, networking configuration, and monitoring. This section outlines the key permissions required for each service in the EKS ecosystem and provides a sample policy for reference, enabling efficient and secure cluster management.

IAM Administrative Access

- Your AWS account has **IAM Administrative Access** to delegate permissions if necessary.
- Terraform CLI is installed and configured with the required AWS CLI credentials.

IAM Permissions

The following permissions are required to create roles, policies, and associate them with EKS resources.

- iam:CreateRole
- iam:AttachRolePolicy
- iam:CreatePolicy
- iam:PutRolePolicy
- iam:GetRole
- iam>DeleteRole
- iam:DetachRolePolicy
- iam>DeletePolicy
- iam:ListRolePolicies
- iam:PassRole (required for EKS to assume roles)

EKS Permissions

These are specific to EKS cluster creation and management.

- eks:CreateCluster
- eks:UpdateClusterConfig
- eks:DescribeCluster
- eks>DeleteCluster
- eks:ListClusters
- eks:CreateNodegroup
- eks>DeleteNodegroup
- eks:ListNodegroups
- eks:UpdateNodegroupConfig
- eks:DescribeNodegroup
- eks:ListUpdates
- eks:DescribeUpdate

EC2 Permissions

Needed for creating networking resources like subnets, security groups, and EC2 instances for worker nodes.

- ec2:CreateVpc
- ec2:DescribeVpcs
- ec2>DeleteVpc
- ec2:CreateSubnet
- ec2:DescribeSubnets
- ec2>DeleteSubnet
- ec2:CreateSecurityGroup
- ec2:AuthorizeSecurityGroupIngress
- ec2:DescribeSecurityGroups
- ec2>DeleteSecurityGroup
- ec2:DescribeInstances
- ec2:TerminateInstances
- ec2:RunInstances
- ec2:CreateTags
- ec2>DeleteTags

Elastic Load Balancer (ELB) Permissions

Required for Kubernetes services using LoadBalancers.

- elasticloadbalancing:CreateLoadBalancer
- elasticloadbalancing:DescribeLoadBalancers
- elasticloadbalancing>DeleteLoadBalancer
- elasticloadbalancing:DescribeTargetGroups
- elasticloadbalancing>CreateTargetGroup
- elasticloadbalancing>DeleteTargetGroup
- elasticloadbalancing:RegisterTargets

Auto Scaling Permissions

For managing worker nodes in managed or self-managed node groups.

- autoscaling:CreateAutoScalingGroup
- autoscaling:UpdateAutoScalingGroup
- autoscaling>DeleteAutoScalingGroup
- autoscaling:DescribeAutoScalingGroups

CloudWatch Permissions

For logging and monitoring EKS events and resources.

- logs:CreateLogGroup
- logs:CreateLogStream
- logs:PutLogEvents
- logs:DescribeLogGroups
- logs>DeleteLogGroup

STS Permissions

Required for role assumption and token-based authentication.

- sts:AssumeRole

Example Policy

Here is an example policy that grants the above permissions:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "iam:*",
        "eks:*",
        "ec2:*",
        "elasticloadbalancing:*",
        "cloudformation:*",
        "autoscaling:*",
        "logs:*",
        "sts:AssumeRole"
      ],
      "Resource": "*"
    }
  ]
}
```

Separation of Installation-Time and Runtime AWS Privileges

There are two distinct phases requiring AWS permissions:

1. Installation-Time (Terraform Provisioning)

- Full administrative AWS IAM privileges are required to provision networking, EKS clusters, node groups, load balancers, storage, DNS, and associated infrastructure. These permissions are fully defined earlier in this section.

2. Runtime (Fabric Operation and Space Provisioning)

- K2view Agent and Deployer Jobs utilize AWS IRSA (IAM Roles for Service Accounts) to assume roles that provision or interact with AWS resources (e.g., S3, RDS, Keyspaces) during Fabric Space creation and operation.
- These runtime IAM roles are generated dynamically based on space profiles, and their policies are described in Section 5.2 (IAM Policy Permissions).

Agent AWS IAM Role (optional)

If configured, the K2view Agent itself may assume an AWS IAM role via IRSA for operational access. This is configured by:

- Associating the Agent's Kubernetes Service Account with an IAM Role ARN using the `eks.amazonaws.com/role-arn` annotation.
- Providing AWS credentials and parameters via Helm chart `values.yaml` fields.

5.2. IAM Policy Permissions

Runtime IAM Roles for Kubernetes Deployer Jobs

During space creation, K2view uses Kubernetes Deployer Jobs orchestrated by the Agent to provision AWS resources (S3, RDS, Keyspaces) dynamically for each Fabric Space. These jobs assume IAM Roles assigned via IRSA to the Service Account operating within each Fabric Space namespace.

For each space:

- A dedicated AWS IAM Role is created based on the selected Space Profile.
- This IAM Role is assigned to the namespace Service Account using the `eks.amazonaws.com/role-arn` annotation.
- The Deployer Job uses this Service Account to invoke AWS APIs with the required permissions.

The policy permissions assigned to these roles are defined below.

Here is the generated AWS IAM Policy associated with *-fabric IAM Role

S3 Permissions

1. `s3:CreateBucket` - `arn:aws:s3:::*`
2. `s3:DeleteBucket` - `arn:aws:s3:::*`
3. `s3:PutObject` - `arn:aws:s3:::*/*`, `arn:aws:s3:::*`
4. `s3:GetObject` - `arn:aws:s3:::*/*`, `arn:aws:s3:::*`
5. `s3:DeleteObject` - `arn:aws:s3:::*/*`, `arn:aws:s3:::*`
6. `s3:ListBucket` - `arn:aws:s3:::*/*`, `arn:aws:s3:::*`

Amazon Keyspaces (Cassandra) Permissions

1. `cassandra:Alter` - *
2. `cassandra:Create` - *
3. `cassandra:Drop` - *
4. `cassandra:TagResource` - *
5. `cassandra:UntagResource` - *

Amazon RDS Permissions

1. `rds-db:connect` - *
2. `rds:CreateDBCluster` - *
3. `rds:CreateDBInstance` - *
4. `rds>DeleteDBCluster` - *
5. `rds>DeleteDBInstance` - *
6. `rds:ModifyDBCluster` - *
7. `rds:ModifyDBInstance` - *
8. `rds:StartDBCluster` - *
9. `rds:StartDBInstance` - *

10. rds:StopDBCluster - *
11. rds:StopDBInstance - *
12. rds:AddTagsToResource - *
13. rds:RemoveTagsFromResource - *

Kubernetes Resource Permissions

In addition to AWS IAM permissions, Deployer Jobs may also create Kubernetes objects within each Fabric Space namespace, such as:

- Deployments, StatefulSets, Services
- PersistentVolumeClaims (PVCs)
- Secrets and ConfigMaps
- RoleBindings and ServiceAccounts

The Agent executes these Kubernetes operations through its namespace-scoped elevated permissions.

5.3. Cloning the K2view Blueprints

After installing a Git client on your machine, you must “clone” the K2view Blueprints. The blueprints incorporate the Fabric Docker Compose Runtime blueprint.

Select a directory to host the K2view Blueprints (Internet access required). Using a shell, change to your Git directory and run the following command:

```
git clone https://github.com/k2view/blueprints.git
```

5.4. Configuring Your Deployment

Various configurable settings are available when using K2view’s [AWS Blueprint](#) and Terraform. Some of these are noted here. Please review the Terraform variables file for a complete list of configurable values for Amazon Elastic Kubernetes Services (EKS). K2view can provide you with assistance in configuring your environment.

Notable settings include the following:

- Region. The region of the cluster.
- Zones. List of availability zones.
- Domain. The domain name of the Ingress Controller.
- Cluster Name. The name of your cluster.
- Instance types. Set of instance types associated with the EKS Node Group
- EFS enabled flag. Flag to enable or disable the EFS module
- Min/Max/Desired Workers. Values associated with worker nodes you intend to employ for auto-scaling.
- Mailbox ID. The Mailbox ID will be provided to you by K2view.

Network Configuration

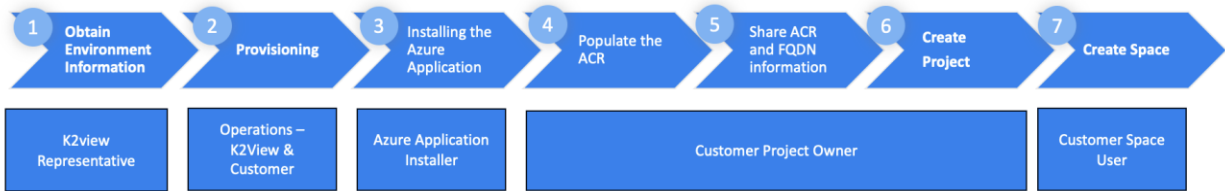
The configurable values of the [Terraform variables file](#) can also be used to configure various settings. Default values are provided. If you have specific needs, please review this configuration file where you will find these.

- Network name. The name of the network.
- VPC CIDR. CIDR range of the network.
- Private subnets. List of private subnet CIDRs.
- Public subnets. List of public subnet CIDRs.

Other Components

For optimal cluster performance and resource management, we leave the choice of Kubernetes auto-scaling tools to the customer. However, we recommend using **Karpenter**, an efficient cluster-level auto-scaler designed to manage nodes dynamically, and the **Horizontal Pod Autoscaler (HPA)** for scaling workloads based on application metrics. These tools can be configured to work together, ensuring seamless scaling of both nodes and pods based on demand. While customers can select alternative auto-scaling solutions, Karpenter and HPA offer a robust combination for maintaining performance and cost-efficiency in dynamic environments.

5.5. Populating Your Container Registry



To populate your container registry, you will need

1. K2view Container registry Account supplied to you by K2view
2. Obtained the list of Docker images to pull from the container registry and push to in consultation with K2view
3. Docker installed on the computer used to pull and push Docker images to your ECR

The [Planning](#) section describes what is necessary to be ready to perform this step.

Commands to Pull Containers from the K2view Container registry

The list of K2view service Docker images will be discussed and recommended to you in consultation with your K2view representative.

Docker tools need to be installed on the computer you are performing this installation. You can download these from <https://docker.com>. The following instructions will “pull” from the K2view Container registry three images of a specific version: fabric-studio, fabric, and the k2-cloud-deployer Docker images.

First, Login

Command:

```
docker login -u [your K2view Container registry username]
https://docker.share.cloud.k2view.com
```

Second, pull the required images. In this example, we pull three.

Commands: In succession, please run the following three commands. Keep in mind that the version and docker images needed may differ.

```
docker pull docker.share.cloud.k2view.com/k2view-images/fabric-studio:8.3.0_291
docker pull docker.share.cloud.k2view.com/k2view-images/fabric:8.3.0_291
docker pull docker.share.cloud.k2view.com/k2view/k2-cloud-deployer:1.8.14
```

Commands to Push Containers to your Container Registry

Now that you have “pulled” images from the K2view Container registry, you can “push” these to your container registry (e.g., [Amazon ECR](#)).

First, obtain the credentials required to perform a Docker Push to your container registry.

Second, perform the Docker Push to your container registry.

Use the Docker login command to login to your container registry

```
docker login -u [your username] [the Container Registry Login server]
```

Third, tag each image you pulled from the K2view Container registry

Tags are used for version control. Tagging Docker images allows you to maintain different versions of the same application, making it easier to roll back to a previous version if needed. This can be especially useful in cases where a new release introduces a bug or breaks compatibility with other components.

In our example, we used the following three commands, respectively, for the fabric-studio, fabric, and k2-cloud-deployer Docker images. Note that the [registry] value is the name used in our example; you will need to use the one associated with your container registry.

```
docker tag docker.share.cloud.k2view.com/k2view-images/fabric-studio:8.3.0_291  
[registry]/k2view-images/fabric-studio:8.3.0_291
```

```
docker tag docker.share.cloud.k2view.com/k2view-images/fabric:8.3.0_291  
[registry]/k2view-images/fabric:8.3.0_291
```

```
docker tag docker.share.cloud.k2view.com/k2view/k2-cloud-deployer:1.8.14  
[registry]/k2view/k2-cloud-deployer:1.8.14
```

Fourth, push images to your container registry

Next, you need to push the Docker images you pulled from the K2view Container registry to your container registry using the command:

```
docker push [image location]
```

In our example, we used:

```
docker push [registry]/k2view-images/fabric-studio:8.3.0_291
```

```
docker push [registry]/k2view-images/fabric:8.3.0_291
```

```
docker push [registry]/k2view/k2-cloud-deployer:1.8.14
```

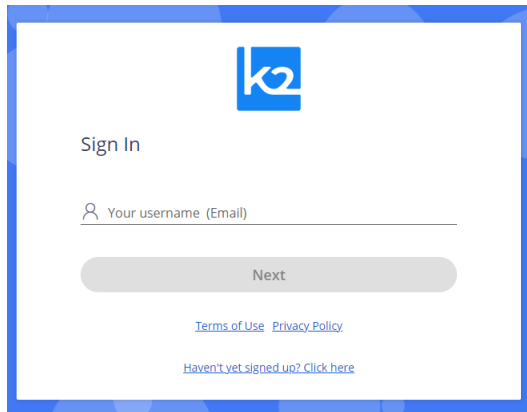
Fifth, please provide the locations you used to K2view

K2view needs these container registry locations of your images to finalize your site's configuration. Please note that you will share these locations with K2view as described in the [Installation Checklist Worksheet](#) section.

5.6. Sign In to K2cloud

To sign in, access cloud.k2view.com using Chrome, Microsoft Edge, or Firefox web browsers.

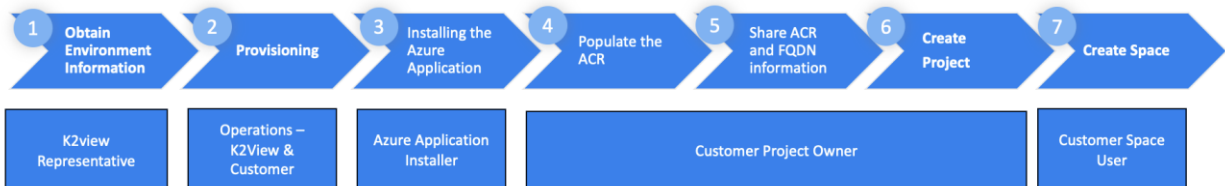
When prompted to sign in, enter your email address. This account needs to have been provisioned for you by K2view. Please refer to the [Prerequisites](#) section.



If provisioned by K2view, you will receive an access code via email you will then enter in the prompt window that is shown when you press Next.

You will be directed to the Spaces page, where you will eventually create spaces. First, you need to create a project.

5.7. Create the Project



You create a project by selecting the Projects top-level navigation tab and pressing “Create Project”. A project requires a name and an associated Git repository. You must provide Git-related information to create a project. To learn more about Git and how to obtain the Repository URL you will use, a branch, and your token, please review the [GitHub](#) section.

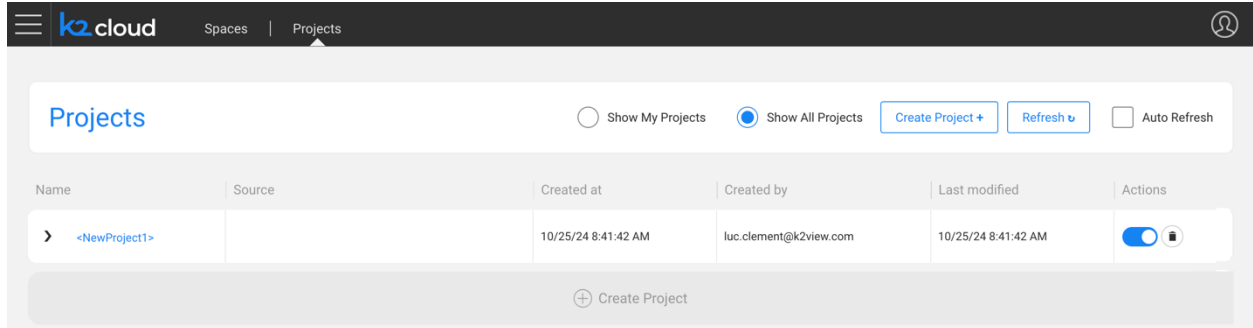
The Fabric spaces you implement are based on a “Space Profile”. You can configure more than one Space Profile per Project. If you need assistance, a K2view representative will assist you. Some of this information will have been shared with you during the [Planning](#) phase.

To create a Space Profile, you need to:

- Give it a name that will have significance to your users
- Select an Image Profile identified in consultation with K2view, and
- Define the site where this space will operate within. K2view will help you define this list.

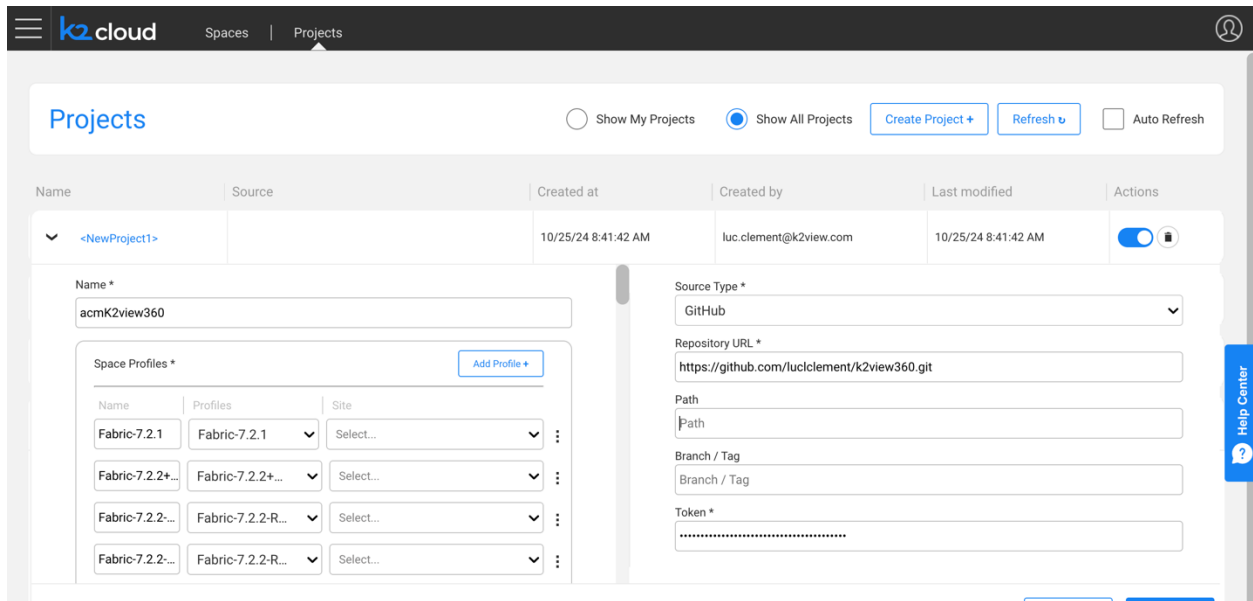
Creating a Project

After pressing Create Project, an empty project will be created.

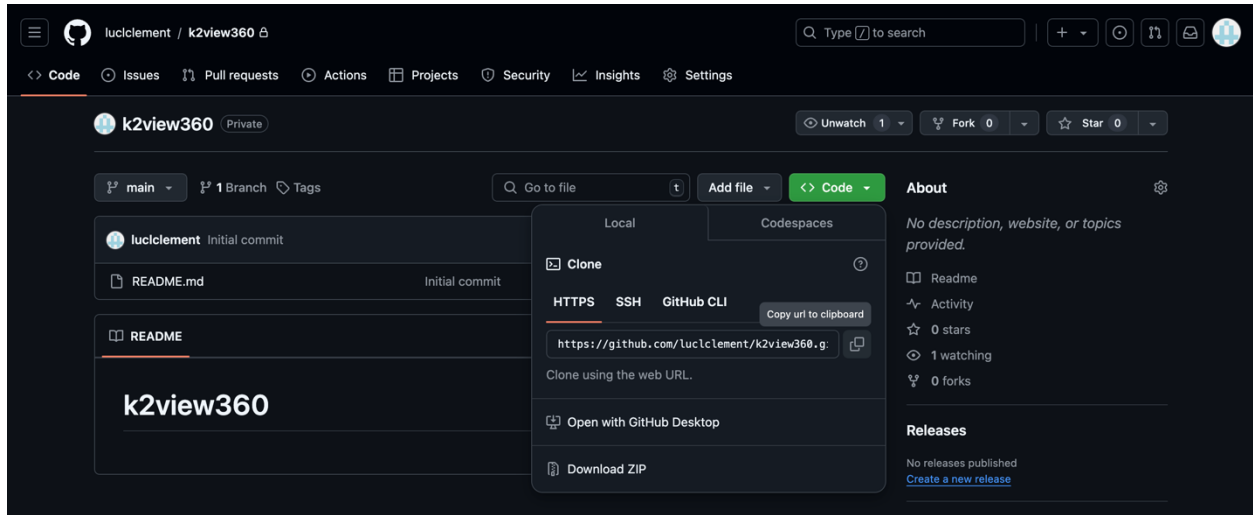


Provide the project with a name and select your space profiles. Again, K2view will have provided this.

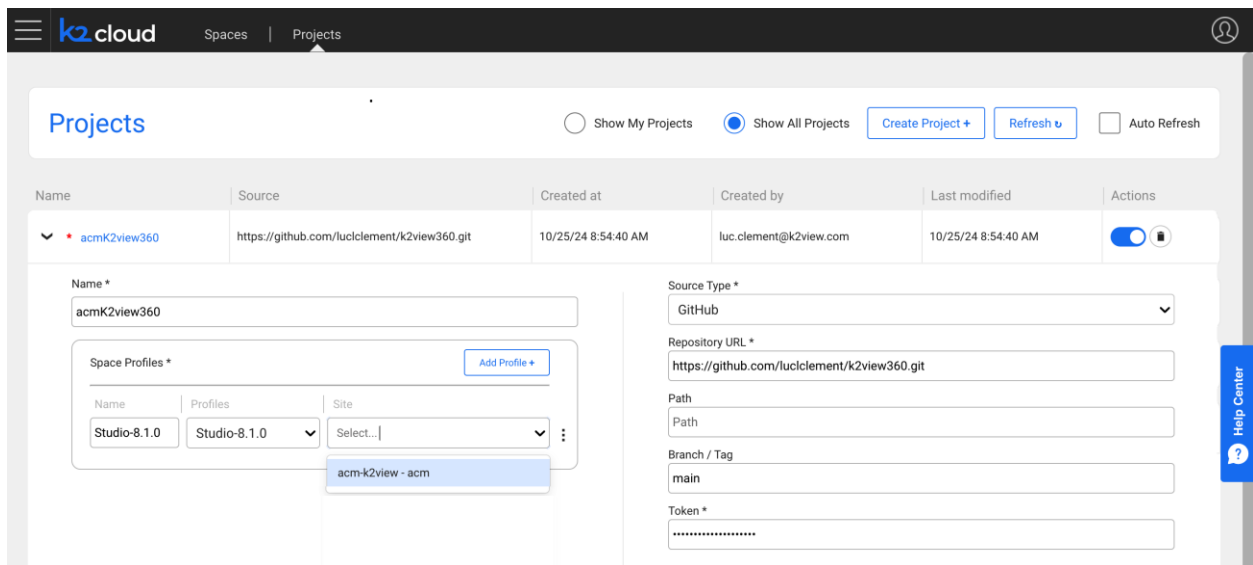
You also need to provide the details of your Git source control system.



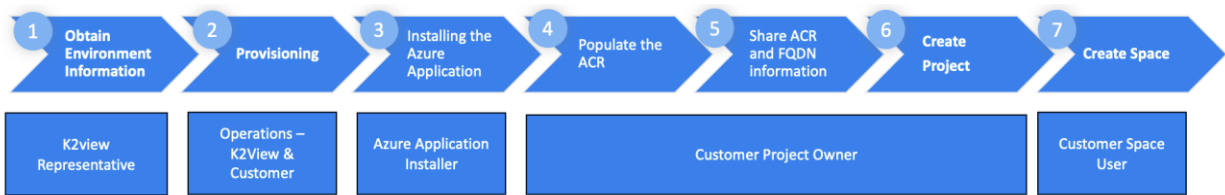
For example, you can obtain the URL from your Git repository, as shown here in our example.



And, finally, please select the site created from the site's dropdown.



5.8. Creating Your First Fabric Space



After your Project is created, you can create a Fabric Space. Here, we show an example of an existing space and a new space being defined.

To create a space

- Give it a name,
- Select the project you created,
- Select your intended profile, one recommended by K2view given your intended purpose, and
- Select site.

If you need to install a K2view extension such as TDM, you must go to your space and install the extension from K2exchange.

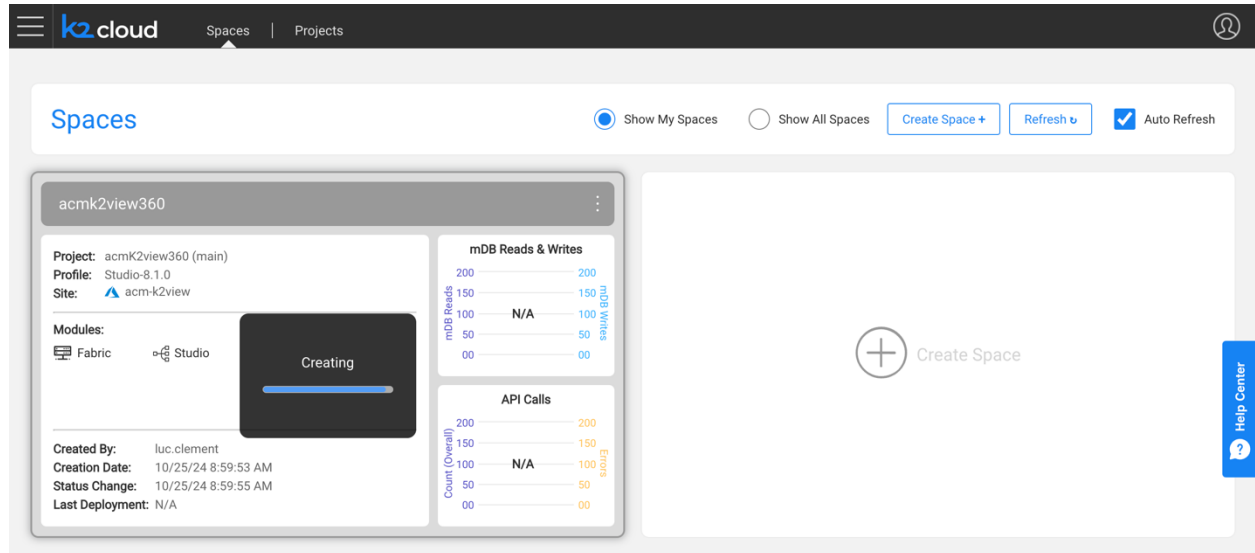
When ready, press Create to trigger the space's creation.

The screenshot shows the K2cloud interface with the 'Spaces' tab selected. A 'New Space' modal is displayed with the following details:

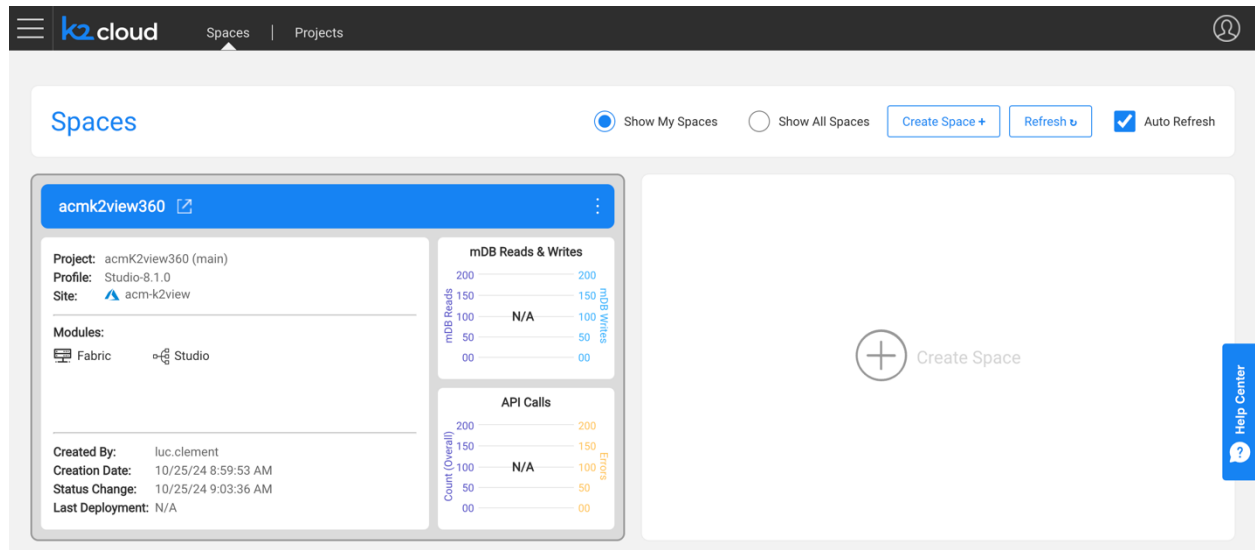
- Space Name ***: acmk2view360
- Project ***: acmk2view360
- Profile ***: Studio-8.1.0
- Site ***: acm-k2view - acm

Buttons for 'Cancel' and 'Create' are at the bottom of the modal. The background shows a list of spaces and a 'Help Center' button on the right.

The Space creation process begins and may take a few minutes to complete.



Once complete your site is displayed to you.



You have successfully created your Project and Space and are ready to begin working with K2view.